

Project 1 - Unix Shell

Đỗ Hoàng Gia Bảo - 23020783

Khoa Điện tử - Viễn Thông

Ngày 22 tháng 3 năm 2026

Tóm tắt nội dung

Báo cáo sẽ trình bày các bước để tạo ra một giao diện shell cho người dùng sử dụng và thực thi các lệnh đó.

Mục lục

1	Nội dung tổng quan bài tập	3
2	Giải bài tập	3
2.1	Đọc dữ liệu đầu vào từ người dùng	3
2.2	Chia câu lệnh thành các phần mã nhỏ (token)	3
2.3	Xử lý chạy lệnh trong tiến trình con	4
2.4	Thực hiện lệnh trong tiến trình con	4
2.5	Tạo tính năng lưu lịch sử output của giao diện	5
2.6	Xử lý điều hướng ">", "<"	5
2.7	Giao tiếp đường ống (pipe)	7

1 Nội dung tổng quan bài tập

Nhiệm vụ của bài tập yêu cầu là thiết kế một giao diện shell có thể chấp nhận lệnh từ người dùng và thực hiện các câu lệnh đó trong một tiến trình khác. Giao diện cần phải hỗ trợ chuyển hướng đầu vào, đầu ra và đường ống để hình thành giao tiếp 2 chiều giữa các cặp lệnh.

2 Giải bài tập

2.1 Đọc dữ liệu đầu vào từ người dùng

Để có thể đọc được dữ liệu người dùng, ta sử dụng hàm `fgets()` để lưu dữ liệu từ người dùng vào một bộ nhớ đệm.

Mã nguồn 1: Tham số hàm `fgets()`

```
fgets(input_buffer, MAX_LINE, stdin)
```

- `input_buffer`: Bộ nhớ đệm để lưu dữ liệu nhập từ người dùng
- `MAX_LINE`: Giới hạn độ dài của câu lệnh
- `stdin`: Luồng dữ liệu đầu vào từ bàn phím

2.2 Chia câu lệnh thành các phần mã nhỏ (token)

Mã nguồn 2: logic chia lệnh thành các mã nhỏ (token)

```
// Split command string into tokens using strtok()
char *token;
int i = 0;

// Get the first token (The command itself)
token = strtok(input_buffer, " ");

while (token != NULL) {
    // Store the pointer to the word in the array
    args[i] = token;
    i++;
}
```

```

        // Get the next token (the arguments)
        // Passing NULL tells strtok to continue with inputBuffer
        token = strtok(NULL, " ");
    }

```

Hàm **strtok** nhận vào chuỗi **input_buffer** và ký tự phân cách là khoảng trắng " ". Nó sẽ tìm khoảng trắng đầu tiên, thay thế nó bằng ký tự kết thúc chuỗi **\0**, và trả về địa chỉ bắt đầu của từ đó.

Ví dụ: Nếu lệnh nhập là **"ls -l -a"**, token lúc này sẽ trở đến chữ **"ls"**.

args[i] = token; Lưu địa chỉ của từ vừa tìm được vào mảng **args**. **token = strtok(NULL, " ");** Khi truyền **NULL** vào tham số thứ nhất, **strtok** hiểu rằng nó cần tiếp tục xử lý chuỗi cũ (**input_buffer**) từ vị trí nó vừa dừng lại.

Vòng lặp sẽ chạy cho đến khi không còn từ nào nữa (hàm trả về **NULL**).

2.3 Xử lý chạy lệnh trong tiến trình con

Để tạo tiến trình con, sử dụng hàm **fork()**

Mã nguồn 3: Sử dụng hàm **fork()** để tạo tiến trình con

```
pid_t pid = fork();
```

- **pid < 0**: tiến trình con không được khởi tạo thành công.
- **pid == 0**: Tiến trình con được khởi tạo thành công.

2.4 Thực hiện lệnh trong tiến trình con

Trong tiến trình con, để có thể thực hiện được lệnh lưu trong **input_buffer**, tiến trình con sử dụng hàm (**execvp()**).

Mã nguồn 4: Sử dụng hàm **execvp()** để thực hiện lệnh

```

// execvp(command, array_of_args)
execvp(args[0], args)

```

- **args[0]**: địa chỉ lưu câu lệnh.
- **args**: Phần lưu các tham số truyền cùng với lệnh.

2.5 Tạo tính năng lưu lịch sử output của giao diện

Khi người dùng nhập "!!", giao diện shell sẽ in ra lịch sử các output được lưu.

Mã nguồn 5: Logic xử lý lệnh "!!"

```
// Check if user has input the history command
if (strcmp(input_buffer, "!!") == 0) {
    if (!has_history) {
        printf("No command in history\n");
        continue;
    }

    // Copy saved history to input buffer and echo the command
    strcpy(input_buffer, history);
    printf("%s\n", input_buffer);
} else {
    // Update the history buffer to store the new command
    strcpy(history, input_buffer);
    has_history = 1;
}
```

Hàm **strcmp** (String Compare) so sánh nội dung người dùng vừa nhập (**input_buffer**) với chuỗi "!!".

- Nếu chưa có lịch sử, chương trình sẽ báo lỗi **No command in history**.
- Nếu có lịch sử, sử dụng hàm **strcpy** để sao chép lịch sử trong bộ nhớ đệm **history** vào **input_buffer**
- Nếu nhập lệnh bình thường, bộ đệm lịch sử sẽ được cập nhật để lưu lệnh đó.

2.6 Xử lý điều hướng ">", "<"

Để có thể xử lý điều hướng, cần phải ghi hoặc tạo file và đọc, viết nó.

Mã nguồn 6: Logic xử lý điều hướng

```
for (int i = 0; args[i] != NULL; i++) {
    // Check for output redirection ">"
    if (strcmp(args[i], ">") == 0) {
        char *filename = args[i + 1];
```

```

        // Open file: Write only. Create if Missing. Truncate if
        exists
        int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644)
        ;
        if (fd < 0) {
            perror("Error opening output file\n");
            return 1;
        }
        dup2(fd, STDOUT_FILENO); // Redirect STDOUT to file
        close(fd); // Close original descriptor
        args[i] = NULL; // Chop the args array here
        break;
    }
    // Check for input redirection "<"
    else if (strcmp(args[i], "<") == 0) {
        char *filename = args[i + 1];
        int fd = open(filename, O_RDONLY); // Open for reading
        if (fd < 0) {
            perror("Error opening input file\n");
            return 1;
        }
        dup2(fd, STDIN_FILENO); // Redirect STDIN to file
        close(fd); // Close original descriptor
        args[i] = NULL; // Chop the args array here
        break;
    }
}
}

```

- **Tìm kiếm toán tử:** Vòng lặp for duyệt qua danh sách tham số để xác định người dùng muốn điều hướng đầu vào (<) hay đầu ra (>).
- **Mở tệp (open):**
 - Với điều hướng đầu ra: Tệp được mở với các cờ O_WRONLY (chỉ ghi), O_CREAT (tạo nếu chưa có) và O_TRUNC (xóa nội dung cũ). Quyền truy cập 0644 cho phép chủ sở hữu đọc/ghi và người khác chỉ đọc.
 - Với điều hướng đầu vào: Tệp được mở ở chế độ O_RDONLY (chỉ đọc).

- **Chuyển hướng luồng (dup2):** Hàm `dup2(fd, STDOUT_FILENO)` hoặc `dup2(fd, STDIN_FILENO)` sẽ ánh xạ tệp vừa mở vào luồng chuẩn tương ứng của hệ thống.
- **Làm sạch tham số:** Lệnh `args[i] = NULL` cực kỳ quan trọng để ngăn hàm `execvp` coi toán tử điều hướng và tên tệp là các đối số của lệnh chính.

2.7 Giao tiếp đường ống (pipe)

Cơ chế đường ống (*Pipes*) cho phép thiết lập giao tiếp giữa hai tiến trình theo nguyên lý *Producer-Consumer*.

Mã nguồn 7: Logic xử lý giao tiếp đường ống

```
// Process pipe logic
if (pipe_index != -1) {
    int fd[2];
    if (pipe(fd) == -1) {
        perror("Pipe_ failed");
        return 1;
    }

    pid_t p2 = fork();

    if (p2 < 0) {
        perror("Fork_ failed");
        return 1;
    }

    if (p2 == 0) {
        // GRANDCHILD: Runs the first command
        dup2(fd[1], STDOUT_FILENO); // Redirect stdout to pipe
        write-end
        close(fd[0]); // Close unused read-end
        close(fd[1]); // Close write-end after dup
        execvp(args[0], args);
        exit(1);
    } else {
        // CHILD: Runs the second command
        dup2(fd[0], STDIN_FILENO); // Redirect stdin to pipe read-
        end
    }
}
```

```
        close(fd[1]); // Close unused write-end
        close(fd[0]); // Close read-end after dup
        execvp(args[pipe_index + 1], &args[pipe_index + 1]);
        exit(1);
    }
}
```

- **Khởi tạo đường ống:** Hàm `pipe(fd)` tạo ra hai bộ mô tả tệp: `fd[0]` để đọc và `fd[1]` để ghi.
- **Tách tiến trình:** Shell thực hiện `fork()` thêm một lần nữa. Tiến trình cháu thực thi lệnh đứng trước dấu ống dẫn, tiến trình con thực thi lệnh đứng sau.
- **Kết nối luồng dữ liệu:**
 - Tiến trình cháu sử dụng `dup2` để ánh xạ `STDOUT` vào `fd[1]`.
 - Tiến trình con sử dụng `dup2` để ánh xạ `STDIN` vào `fd[0]`.
- **Đóng mô tả tệp thừa:** Sau khi ánh xạ, các bản sao cũ của `fd[0]` và `fd[1]` cần được đóng ngay lập tức để hệ thống có thể gửi tín hiệu kết thúc tệp (EOF) một cách chính xác.